OPAL AI

Build Your Own AI Assistant

The complete beginner's guide to OpenClaw

Before you start: import this guide into Claude or ChatGPT

We highly recommend this because code is specific — one wrong character can break a step. Paste what's in your Terminal into an AI and it will troubleshoot with you in real time. Once your OpenClaw is running, you can also ask it to help you through the rest of this guide — that's exactly what it's there for.

Upload this PDF, then paste this prompt:

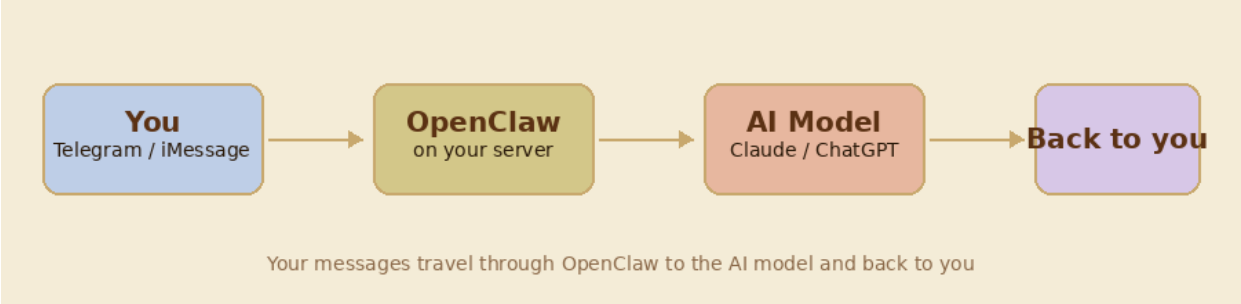
```
I am setting up OpenClaw for the first time. I have uploaded the setup guide.
Please stick to the steps in this guide and do not suggest shortcuts or
alternative methods — I want to follow it exactly as written. If something
goes wrong or I get stuck, help me troubleshoot it step by step, going slowly
and explaining what each fix does. Do not move on to the next step until I
confirm the current one worked.
```

Keep this chat open the whole time you work through the guide.

What you are building

By the end of this guide you will have a personal AI assistant that:

- Lives on a rented server on the internet, running 24/7 even when your laptop is closed
- Talks to you through Telegram, iMessage, WhatsApp, or another channel of your choice
- Can manage your email, save notes, and run tasks on your behalf
- Costs about $7–$10 USD per month to keep online



Your messages travel through OpenClaw to the AI and back to you.

Part 1 — Before you start

What is OpenClaw?

OpenClaw is free, open-source software that turns a rented computer into your own personal AI agent. Think of it as the engine running in the background — it connects your messaging app (Telegram, iMessage, WhatsApp) to a powerful AI model like Claude or ChatGPT, and makes sure everything keeps running around the clock even when you are not at your laptop.

The difference between using OpenClaw and just opening ChatGPT in a browser tab is this: ChatGPT forgets you the moment you close the window. OpenClaw runs on a server that never turns off, remembers everything you tell it to remember, and can take actions on your behalf — managing your inbox, tracking your priorities, and sending you a morning brief — all without you having to open an app.

In plain English:
ChatGPT or Claude in a browser tab = a conversation that forgets you when you close it.
OpenClaw = a personal assistant that lives on your own server, stays on 24/7, remembers everything, and texts you back on whatever app you already use.

Take a breath. A lot of beginner OpenClaw content makes people feel like they need to be half-engineer, half-devops wizard, half-guy-on-YouTube-with-six-monitors. You do not.

We know this process can feel daunting if you're not technical — but once you set it up, all you'll need to do is text your AI. Every command below is explained in plain English.

One thing to have ready now:
Open a notes document somewhere you trust — Notes, Notion, a Google Doc, anything. We will tell you exactly when you need to save a key, IP address, or password. You don't need to prepare anything else before starting.

Part 2 — Create your server

A VPS (Virtual Private Server) is a rented computer on the internet. OpenClaw lives there so it stays online all day and night. When your laptop sleeps, the server does not.

We recommend Hetzner Cloud. It's reliable, reasonably priced, and beginner-friendly. Your server will cost about $7–$10 USD per month.

Step 1 — Create your Hetzner account

1. Go to console.hetzner.cloud and sign up
2. Confirm your email address and log in

Step 2 — Generate an SSH key on your laptop

An SSH key is how your laptop proves it is allowed to enter your server. Think of it like a digital key — one half stays on your laptop, the other gets uploaded to Hetzner.

On a Mac:

3. Press Command + Space, type Terminal, press Enter
4. Type the following exactly and press Enter:

```
ssh-keygen -t ed25519
```

Press Enter through all the prompts. The defaults are fine.

5. Now print your public key by typing:

```
cat ~/.ssh/id_ed25519.pub
```

6. You will see a long line beginning with ssh-ed25519. Copy that entire line and paste it into your notes.

On Windows:

7. Open PowerShell (search for it in the Start menu)
8. Run the same commands as above

Step 3 — Add your SSH key to Hetzner

9. In the Hetzner console, go to Security > SSH Keys > Add SSH Key
10. Paste the public key you just copied (the long line starting with ssh-ed25519)
11. Give it a label like "My Laptop" and save it

Important: only paste the public key. Never paste the private key. The private key never leaves your laptop.

Step 4 — Create the server

12. In Hetzner, click + New Project and name it OpenClaw
13. Click Add Server and use these exact settings:

Setting	Value
Location	Closest to you (e.g. Ashburn for US East, Helsinki for Europe)
Image	Ubuntu 24.04
Type	CPX21 or CPX31 — 4GB+ RAM, roughly $7–$10/month
SSH Key	Select the key you just added
Name	See the naming note below

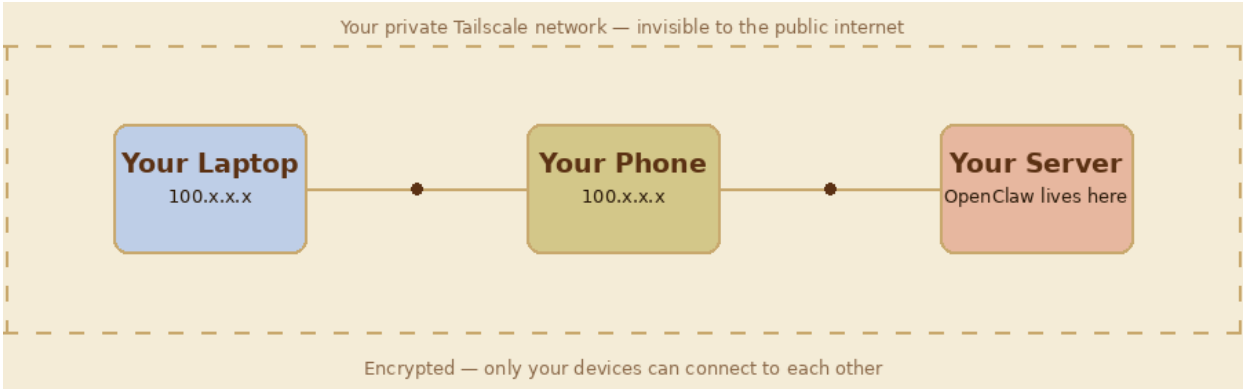
Name your AI assistant!

Before you create the server, pick a name for your assistant. Some ideas: Luna, Sage, Aria, Pepper, Nova, Iris. Once you've chosen, name the Hetzner machine [yourname-openclaw] — for example: luna-openclaw. Save the name in your notes.

14. Click Create & Buy Now. Your server will be ready in about 30 seconds.
15. Once created, you will see your server listed in the Hetzner dashboard. Click on it. On that page you will see a Public IP address — a number like 178.156.253.141. Copy that into your notes. Label it "Server IP".

Part 3 — Connect privately with Tailscale

Tailscale creates a private, encrypted network between your laptop and your server. Instead of your server being exposed to the entire internet, only your own devices can reach it.

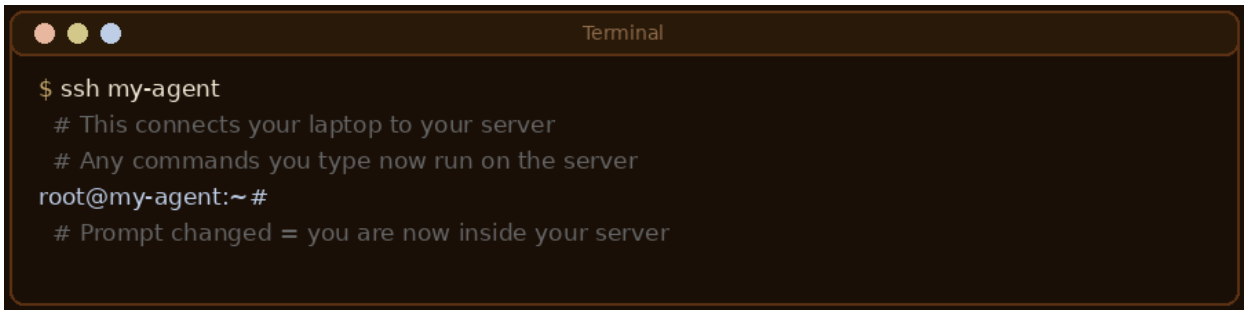


Your laptop, phone, and server all join a private network — invisible to everyone else.

Step 1 — SSH into the server for the first time

First we need to explain what "SSH into your server" means, because we will say this a lot.

What "SSH into your server" means:
SSH is the secure way your laptop remotely controls the server. When we say "SSH in", we mean: open Terminal on your laptop, type a command, and your screen will now be showing what's happening inside your server instead of your laptop. Any commands you type will run on the server.



The Terminal before (on your laptop) and after (inside your server). The prompt changes.

Open Terminal and type:

```
ssh root@YOUR_SERVER_IP
```

Replace YOUR_SERVER_IP with the IP you saved in your notes. Press Enter.

The first time you connect, Terminal asks if you want to continue. Type yes and press Enter.

> Whenever we say "SSH into your server" from now on, this is what we mean: open Terminal and type ssh root@YOUR_SERVER_IP (or later, ssh my-agent after Step 5 below).

Step 2 — Update the server

Run these two commands one at a time. Wait for each to finish.

```
sudo apt update
sudo apt upgrade -y
```

If Ubuntu asks whether to restart a service, choose the recommended answer.

Step 3 — Install Tailscale on the server

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

This prints a URL. Open it in your browser and sign in to your Tailscale account. Create a free account at tailscale.com if you don't have one. This authorises the server to join your private network.

Step 4 — Install Tailscale on your laptop

16. Download Tailscale from tailscale.com/download and install it
17. Sign in with the same Tailscale account you just used on the server
18. Go to login.tailscale.com/admin/machines and find your server in the list. You will see a Tailscale IP address starting with 100.x.x.x — copy that into your notes and label it "Tailscale IP"

Step 5 — Set up a shortcut so reconnecting is easy

On your laptop, open Terminal and run:

```
nano ~/.ssh/config
```

This opens a simple text editor. Add the following lines (replace YOUR_TAILSCALE_IP with the 100.x.x.x address from your notes, and my-agent with your assistant's name):

```
Host my-agent
    HostName YOUR_TAILSCALE_IP
    User root
```

Press Control+X, then Y, then Enter to save.

Step 6 — Test the Tailscale connection

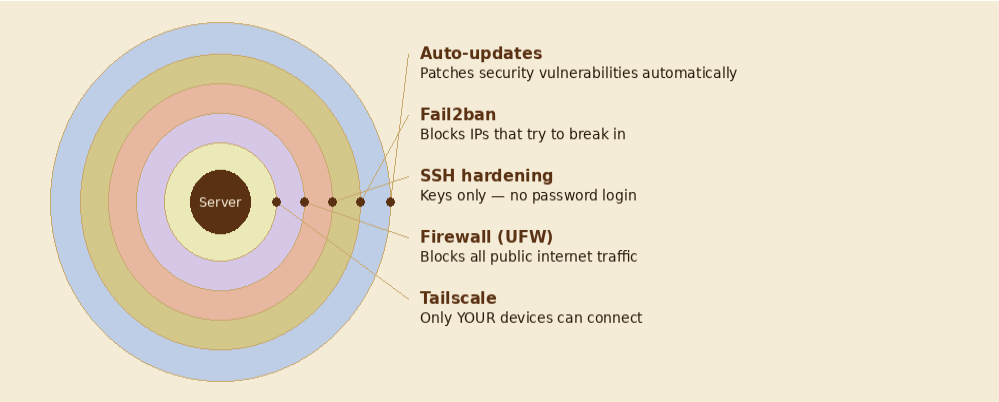
Exit your current session (type exit), then reconnect using your shortcut:

```
ssh my-agent
```

Why you will keep using this command:
From now on, typing ssh my-agent is how you get back into your server whenever you need to do something for your AI assistant — like giving it access to secret phrases, passwords, or new tools. Your assistant lives on that server, and this command opens the door to where it lives. You will come back here more than you might expect.

Part 4 — Lock down the server

Your server is on the internet, and automated bots start scanning it within minutes of creation. Don't worry — we're going to put up several layers of protection that block all of that. Think of it like installing a series of locks on your front door.



Each layer independently blocks a different type of threat.

SSH into your server first (type ssh my-agent in Terminal), then run each block of commands:

Firewall — blocks all public traffic

This tells the server to only accept connections from your private Tailscale network and refuse everything else from the internet.

```
ufw allow in on tailscale0
ufw allow out on tailscale0
ufw default deny incoming
ufw default allow outgoing
echo 'y' | ufw enable
```

After running this, the public IP will no longer work for SSH. From now on, always connect using ssh my-agent (the Tailscale route). That's a feature, not a problem.

Disable password login — keys only

This makes it so only your laptop's SSH key can log in — no guessable passwords.

```
echo 'PasswordAuthentication no' > /etc/ssh/sshd_config.d/hardening.conf
echo 'KbdInteractiveAuthentication no' >>
/etc/ssh/sshd_config.d/hardening.conf
systemctl reload ssh
```

Automatic security updates

This keeps your server automatically patched so you don't have to think about it.

```
apt-get install -y unattended-upgrades
tailscale set --auto-update
```

Part 5 — Install OpenClaw

Make sure you are SSH'd into your server (open Terminal and type ssh my-agent) before running any of these commands. Remember: when we say "SSH into your server", that's always what we mean.

Step 1 — Install Node.js

OpenClaw needs Node.js to run. Install it with:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt-get install -y nodejs
node --version
```

The last command should print v22.x.x or higher. If it does, continue.

Step 2 — Install OpenClaw

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

The installer will run for a while. Let it finish. If you see a network error, wait 30 seconds and try again.

Step 3 — Confirm it worked

```
openclaw --version
```

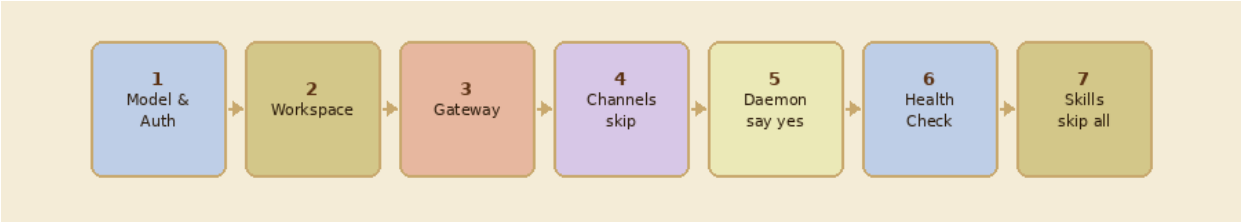
If this prints a version number, you are good to go.

> If it says "command not found": type exit, reconnect with ssh my-agent, and try openclaw --version again. If it still fails, paste the error into Claude or ChatGPT and ask for help.

Part 6 — Run the onboarding wizard

This is the guided setup that configures everything: your AI model, the gateway, your messaging channels, and the background service that keeps it running 24/7.

```
openclaw onboard --install-daemon
```



The onboarding wizard walks you through these steps in order.

Step	What to do
1. Model & Auth	Choose your AI provider. We recommend selecting ChatGPT Codex — it lets you sign in with your existing ChatGPT account (a paid account is required). This means no API key to set up.
2. Workspace	Where your agent stores files. Press Enter to accept the default.
3. Gateway	The local server that handles messages. Accept all defaults.
4. Channels	Skip this step. We will walk you through connecting your messaging app in Part 7.
5. Daemon	Say yes. This installs a service so your agent runs 24/7 and restarts automatically if the server reboots.
6. Health Check	The wizard checks the gateway started successfully.
7. Skills	Skip everything. You can come back and add skills later once your agent is running.

Introducing yourself to your assistant

After onboarding finishes, OpenClaw will say hello in the Terminal and ask you a few questions: who it is, who you are, and how it should behave. This is your chance to give it a name and a personality.

Some things you can tell it:

- Your name and what you do for work
- Things you want it to help with most (email, notes, scheduling, research)
- The vibe you want — professional, casual, witty, warm
- Its name, if you want it different from the machine name

- Its emoji — OpenClaw will let you pick one that appears in its messages

Example intro you could paste:
"My name is [your name]. I work in [your field]. I want you to help me manage my email, keep track of my priorities, and be a second brain for my work and personal life. Be warm, direct, and a little witty. Your name is Luna and your emoji is a crescent moon."

Part 7 — Connect your messaging app

This is where your assistant becomes real. You will connect it to the app you want to text it through. We support several options — pick the one that works best for you.

Option A — Telegram (recommended)

Telegram is our top recommendation because it supports Topics — separate threads inside one group chat for different areas of your life. You can have a Work topic, a Grocery List topic, an Ideas topic, all in one place without things getting mixed up.

Step 1 — Create your bot with BotFather

19. Open Telegram on your phone
20. Search for @BotFather and start a chat with it
21. Send the message: /newbot
22. BotFather will ask for a display name (e.g. "Luna") and a username (must end in "bot", e.g. "luna_opal_bot")
23. When finished, BotFather gives you a token — a long string like 123456789:ABC-DEF. Copy it into your notes and label it "Telegram token".

Step 2 — Add the token to OpenClaw

SSH into your server (ssh my-agent), then run:

```
openclaw configure --section channels
```

Select Telegram and paste your bot token when prompted.

Step 3 — Restart the gateway and approve yourself

```
openclaw gateway restart
```

24. Open Telegram and send any message to your new bot (try: hello)
25. The bot replies with a pairing code because it does not know you yet
26. Back in the server Terminal, approve yourself:

```
openclaw pairing list telegram
openclaw pairing approve telegram CODE
```

Replace CODE with the code the bot sent. Codes expire after 60 minutes.

27. Send your bot another message. This time you will get a real response from your assistant.

Set up Telegram topics

Once your bot is replying, set up topics to keep different parts of your life organised.

- Create a new Telegram group and add your bot to it as a member
- In the group settings, turn on Topics
- For now, we recommend creating two topics: General (already exists) and Inbox
- The Inbox topic is for your email assistant — we will set that up in Part 10
- Later you can add topics for anything: Work, Grocery List, Ideas, Travel, Finance

Tip: keep each topic single-purpose. Don't mix your grocery list with your work projects — your assistant works better when things are separated.

If your bot replies in DM but not in the group

This happens sometimes. If your assistant responds when you message it directly but stays silent in the Telegram group, go back to Terminal, SSH into your server, and talk to your OpenClaw directly there — explain the issue and ask it to troubleshoot and fix it.

You can always talk to your assistant directly in Terminal:
If Telegram, iMessage, or WhatsApp stops working for any reason, you can always SSH into your server and interact with your agent there directly while you fix things. Type:

```
openclaw chat
```

This opens a direct chat session in your Terminal. Use it to troubleshoot, give your agent new instructions, or just talk to it while a messaging channel is being fixed.

Option B — iMessage

You can connect OpenClaw to iMessage so your assistant texts you like a contact on your phone.

iMessage setup requires a Mac and an Apple ID. SSH into your server and run: openclaw configure --section channels, then select iMessage and follow the prompts. Your assistant will walk you through each step.

Option C — WhatsApp

WhatsApp integration connects your assistant to your existing WhatsApp account.

SSH into your server and run: openclaw configure --section channels, then select WhatsApp and follow the prompts. You will be asked to scan a QR code with your phone's WhatsApp app.

Other options (Slack, Discord, and more)

OpenClaw also supports Slack, Discord, and other messaging platforms. If you want to connect one of those, ask Claude or ChatGPT for step-by-step instructions — just tell it which app and that you are using OpenClaw.

Part 8 — Health checks

Even if your first message worked, run these checks. A one-time success is not the same as a sturdy setup.

SSH into your server and run:

```
openclaw status
openclaw gateway status
openclaw health
openclaw doctor
openclaw channels status --probe
```

Healthy signals to look for:

- Runtime: running
- Doctor shows no blocking issues
- Your chosen channel (Telegram, iMessage, etc.) shows as connected or ready

> If anything looks wrong: paste the output into Claude or ChatGPT and ask it to help you interpret and fix it.

Part 9 — Set up memory and planning

Your assistant has no memory between conversations by default. This step gives it a proper system so it remembers your priorities, tracks what's in progress, and sends you a morning brief.

Now that your messaging channel is connected, you can send this prompt directly through Telegram, iMessage, or WhatsApp (or paste it into the Terminal chat using openclaw chat if you prefer). Copy and paste the whole thing:

Copy and paste this into your assistant:

```
Set up a robust memory + planning system for me in this workspace.

Goal: I want continuity across days, no repeated stale blocker messages, and
clear daily execution updates.

Please do this now without asking permission first:

1. Create or verify these files and folders:
   /root/.openclaw/workspace/memory/
   /root/.openclaw/workspace/memory/state-of-play.md
   /root/.openclaw/workspace/memory/YYYY-MM-DD.md (daily logs)
   /root/.openclaw/workspace/knowledge/PARA/00-priorities.md
   /root/.openclaw/workspace/knowledge/PARA/00-blockers.md
   /root/.openclaw/workspace/MEMORY.md

2. Operating rules:
   - state-of-play.md is highest-priority truth unless a newer dated entry
overrides it
   - Never resurface blockers already marked resolved
   - After any meaningful progress, append to today's daily memory: what
changed, decisions made, what got unblocked, next action
   - Keep priorities and blockers files current
   - Only promote durable high-signal insights into MEMORY.md

3. Set up a morning brief at 9am with: top 3 priorities, critical unresolved
blockers, recently unblocked items, deadlines and risks, next action you will
take

4. Set up nightly maintenance at 2am to reconcile all memory files

5. Guardrails: no fabricated facts, keep updates concise, prefer newest
evidence, send no update if nothing meaningful to report

After setup, ask me in one message: timezone, morning update time, midday
check-in preference, nightly update style, which chat or topic to send updates
to, top 3 priorities this week, what to never remind me about once resolved,
preferred update tone.
```

Part 10 — Connect Gmail

Email is one of the most powerful things your assistant can help with. We connect it in safe mode: your assistant can read, sort, and draft replies, but it can never send anything without your approval.

Why we do not give it send access: your assistant can help you manage hundreds of emails, but you should always be the one deciding what leaves your inbox under your name. Every draft it writes stays as a draft until you review and send it yourself.

Step 1 — Paste this into your assistant

Send this message through your connected messaging channel (or via openclaw chat in Terminal):

Copy and paste this to your assistant:

```
Help me connect my Gmail account so you can help manage my inbox.

Please walk me through each of these steps:
1. Set up a Google Cloud project and enable the Gmail API
2. Configure the OAuth consent screen
3. Create OAuth client credentials
4. Request only these Gmail scopes: read, modify/organise, compose/draft
   Do NOT request send scope
5. Complete the OAuth flow and confirm access with this check:
   OK Read   OK Organise/Modify   OK Draft/Compose   NO Send (must be absent)
   If send scope appears, stop and ask me before continuing.

Then store the credentials securely using the terminal secret flow:
   cd /root/.openclaw/workspace/agent-inbox-mvp
   scripts/secret set GOOGLE_CLIENT_ID
   scripts/secret set GOOGLE_CLIENT_SECRET
   scripts/secret set GOOGLE_OAUTH_REFRESH_TOKEN
   scripts/secret set GOOGLE_OAUTH_ACCESS_TOKEN
   scripts/secret list

Rules: never post secret values in our chat. Never commit secrets to git.
Use scripts/secret for all key updates.

After setup, verify it works by: listing my recent unread emails, applying a
test label to one low-risk email, and creating one reply draft to an important
thread.
Then ask me: which categories to auto-archive, which senders should always
stay in inbox, daily triage time and timezone, draft tone preference, and
unsubscribe mode.
```

Email guardrails that never change

- Your assistant will never auto-send an email
- Every reply stays as a draft until you review and approve it
- If anything risks sending without your approval, your assistant stops and asks you first
- Secret credentials are stored securely in Terminal, never in your chat

Part 11 — Working with your assistant long term

AI agents are genuinely new. They are powerful but imperfect, and yours will need training — just like a new hire in a role they have never done before. In the first few weeks you will correct it, teach it your preferences, and help it understand how you work. The good news: your OpenClaw assistant will help you make it better. It can update its own memory, rewrite its own rules, and fix its own mistakes — as long as you tell it what went wrong.

How to train your assistant

- If it forgets something: say "Fix this so it does not happen again." It will update its own memory files.
- If the memory is not working: say "Check and repair your memory files." It will diagnose and fix it.
- If a skill or feature stops working: paste the error into the chat and say "Diagnose and fix this."
- If you want it to do something new: just ask. It can usually figure out how to set it up itself.
- If you are frustrated: say exactly what went wrong in one sentence. One problem at a time is always faster than "nothing works".

When something goes wrong

Here are the most common things that break, in plain language, with exact prompts you can use:

Problem	What it means	Say to your OpenClaw (via openclaw chat in Terminal)	Paste to Claude or ChatGPT
OpenClaw is not replying to my messages	The gateway service stopped, pairing expired, or there is an API key problem	`openclaw chat Then say: 'You are not replying to my Telegram messages. Diagnose why and fix it.'`	'My OpenClaw assistant stopped replying on Telegram. I am on Ubuntu 24.04, Hetzner, Tailscale. Output of openclaw gateway status: [paste it]. What is wrong and how do I fix it?'
My assistant does not remember things from yesterday	Memory files are not being written or read correctly	`'Your memory is not working. Check your memory files, find the problem, fix it, and confirm it is working.'`	'I use OpenClaw with MEMORY.md and PARA files. My agent is not reading previous memory. Here is what I see: [describe]. What should I try?'
I cannot SSH into my server	Tailscale disconnected or SSH key issue	`Cannot reach assistant — check Tailscale is running on your laptop (look for Tailscale in your menu bar or taskbar)`	'I cannot SSH into my Hetzner server using ssh my-agent. I use Tailscale. Error: [paste exact error]. What should I check first?'
I want it to do something new and it says it cannot	Missing skill, permission, or integration	`'I want you to be able to [task]. What do you need me to set up so you`	'I use OpenClaw. I want my assistant to be able to [task]. What skill, integration, or config

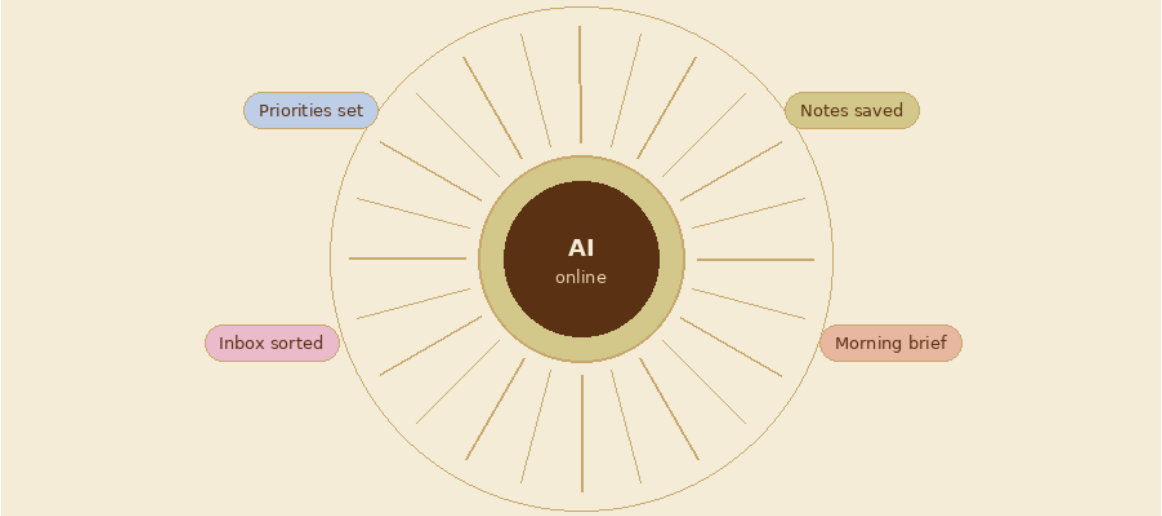
		`can do this?'`	do I need to add?'
It keeps making the same mistake over and over	Missing rule in its memory files	`'You keep doing [thing]. Write a permanent rule so you never do this again. Add it to MEMORY.md now.'`	'I use OpenClaw. My agent keeps doing [thing] even after I told it not to. How do I write a permanent rule into its memory?'

The mindset that gets you far:
Your agent is not broken — it just needs better instructions. Every mistake is free training data. The people who build great AI assistants are not the ones who never hit errors. They are the ones who fix errors and make the agent smarter each time.

Setup checklist

Run through this before calling setup complete:

- I can SSH into the server with ssh my-agent
- openclaw --version returns a version number
- Onboarding finished and I introduced myself to my assistant
- My messaging channel is connected and my assistant replied
- openclaw gateway status looks healthy
- Memory system is set up and my assistant confirmed it
- Gmail is connected in draft-only mode
- I know how to get back into the server (ssh my-agent)
- Important credentials are saved in my notes



Your assistant is live.

Running 24/7. On your server. Under your control.

You just did something most people will never do. You built your own AI — from scratch — without being a developer.

Open your messaging app right now and just talk to it.

A few things to try in your first week:

```
"What are my top 3 priorities today?"
"Summarise my inbox and draft replies to anything urgent."
"I just had a great idea — save it so we can come back to it."
"What did we talk about yesterday?"
"Send me a morning brief tomorrow at 9am."
```

More guides, tips, and drops coming soon at use-opal.com
Questions? Email us at use.opal.ai@gmail.com — we actually reply.